

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion y Diseño Digital

Carrera de Ingeniería de Software

Guía de la Cuarta Entrega (Entrega Final) del Proyecto Fin de Curso (PFC)

Aplicaciones Web — Quinto nivel

Periodo Académico Presencial 2026-2027

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

`gguerrero@uteq.edu.ec`

Etiqueta objetivo de esta entrega: v1.0.0

Fecha de cierre: viernes 17 de agosto de 2026 (semana 17)

Precede a: Defensa oral del PFC (semana 17)

Continúa desde: Entrega 3 (v0.9.0-rc, 24 de julio),

Entrega 1B (v0.7.0, semana del 14 de junio) y

Entrega 1A (v0.3.0, semana del 4 de junio)

Quevedo — Los Rios — Ecuador

2026

1. Naturaleza y ubicacion temporal de la Entrega Final

La Cuarta Entrega es la *Entrega Final* del Proyecto Fin de Curso. Cierra el ciclo iniciado en la Entrega 1A (semana del 4 de junio de 2026), donde se consolido el corpus de ingeniería de requisitos y la arquitectura preliminar; continuado en la Entrega 1B (semana del 14 de junio, v0.7.0) donde se produjo el primer módulo funcional con autenticación JWT y CRUD sobre Spring Data JPA; y elevado al estado de candidato a estable en la Entrega 3 (24 de julio, v0.9.0-rc) con reproducibilidad automática, evidencia empírica preliminar, trazabilidad end-to-end e ingeniería de requisitos formalizada según ISO/IEC/IEEE 29148:2018 [16, 31]. La Entrega Final cierra el 17 de agosto de 2026, semana 17 del calendario académico, y precede a la defensa oral de la misma semana.

Con la Entrega Final el equipo produce simultaneamente tres artefactos indisociables. Primero, el *sistema software* en su versión estable v1.0.0, con la aplicación web completa (backend Java 21 LTS con Spring Boot 3.2.x, frontend Angular 17+, base de datos PostgreSQL 16 con migraciones Flyway, cache Redis 7 y orquestacion Docker Compose), con toda la evidencia empírica cuantitativa y reproducible que respalda cada una de sus propiedades, y **desplegado en un ambiente accesible publicamente** durante el periodo de defensa. Segundo, el *documento tecnico academico final del PFC*, redactado con el rigor, la estructura y la trazabilidad exigidos por la comunidad científica internacional en ingeniería de software y en ingeniería de requisitos [31, 40, 16, 30, 41]. Tercero, el *paquete de datos, materiales y metadatos* que hace al conjunto verificable, reproducible y reutilizable por cualquier tercero sin comunicación adicional con los autores [42, 35, 29, 1].

La Entrega Final no es una repeticion ampliada de la Entrega 3. Su naturaleza es cualitativamente distinta. Mientras la Entrega 3 verifica que el sistema es tecnicamente reutilizable por un tercero, la Entrega Final verifica que todo el trabajo del equipo esta narrado, argumentado y documentado siguiendo las mismas convenciones que un lector experto de una revista científica de alto impacto espera encontrar en un articulo original de investigación aplicada. La diferencia entre ambas es la diferencia entre entregar un artefacto que funciona y entregar un artefacto que ademas *explica por que funciona, con que evidencia, bajo que condiciones, con que amenazas a la validez y en que se distingue de lo publicado previamente* sobre problemas equivalentes [14, 28, 31].

Por esta razón, la Entrega Final consolida un cuarto componente indisociable: la *aplicacion integra de todas las observaciones acumuladas* en las tres entregas previas. Cada observación recibida en los informes de retroalimentación de las Entregas 1A, 1B y 3 debe estar resuelta y su resolución trazable en el historial de commits, con referencia explícita en la bitacora docs/observaciones/OBSERVACIONES.md. Esta bitacora acumulativa es la evidencia primaria de que el equipo produce software académico maduro y no una simple sucesion de entregables aislados.

La lista siguiente resume el arco completo de las cuatro entregas del PFC para situar visualmente la Entrega Final.

Arco completo de entregas del PFC (calendario académico 2026)

- **Entrega 1A** (semana del 4 de junio, v0.3.0): consolidación del corpus de ingeniería de requisitos (SRS conforme a ISO/IEC/IEEE 29148:2018, casos de uso, historias de usuario, requisitos no funcionales), diseño arquitectónico C4 nivel 1 y 2, seleccion y justificación de la pila, esqueleto ejecutable con Docker [16, 7, 8, 6].
- **Entrega 1B** (semana del 14 de junio, v0.7.0): primer módulo funcional. Módulo de autenticación stateless con JWT (jjwt 0.12, cookie `HttpOnly + Secure + SameSite=Strict`), CRUD completo de al menos una entidad principal con Spring Data JPA sobre PostgreSQL 16, migraciones Flyway 9.x, cache Redis 7 para blacklist de JTI revocados, Springdoc OpenAPI 2.x, Angular 17+ con interceptor JWT, docker-compose con healthcheck, mínimo cinco pruebas JUnit 5 + MockMvc verdes [18, 22, 39, 25].
- **Entrega 3** (24 de julio, v0.9.0-rc): aplicación de observaciones acumuladas, consolidación de la ingeniería de requisitos con SRS ISO/IEC/IEEE 29148:2018, reproducibilidad automática, evidencia empírica cuantitativa preliminar, trazabilidad end-to-end completa y publicabilidad del artefacto (FAIR, DOI Zenodo, CITATION.cff, CRediT) [16, 42, 1].
- **Entrega Final** (17 de agosto, semana 17, *esta guía*, v1.0.0): sistema estable con cobertura JaCoCo $\geq 70\%$, k6 y Lighthouse dentro de umbrales, auditoría completa de los seis controles OWASP mínimos, puesta en producción en ambiente accesible, documento académico final con estructura IMRaD ampliada e ingeniería de requisitos culminada, paquete de datos con DOI propio, y sistema listo para archivo permanente [25, 24, 31].

2. Requisitos ineludibles de la Entrega Final

Los requisitos que siguen se organizan en siete bloques (A a G). El equipo cierra los siete bloques al mismo tiempo. La Entrega Final no admite entregas parciales: la ausencia íntegra de un bloque impide la evaluación del conjunto.

2.1. Bloque 0 — Aplicación íntegra de las observaciones acumuladas

Antes de abordar los bloques técnicos de la Entrega Final, el equipo cierra la deuda de calidad acumulada durante las tres entregas anteriores. Este bloque no es opcional y su verificación es previa a la calificación de cualquier otro bloque.

0.1 — Bitácora acumulativa de observaciones y su resolución

- El archivo `docs/observaciones/OBSERVACIONES.md`, iniciado en la Entrega 3, se completa con las observaciones recibidas en el informe de retroalimentación de la propia Entrega 3, siempre con código único continuando la numeración (`OBS-XX`, `OBS-XY`, ...).

- Para cada observación se declara: fuente (Entrega 1A, 1B o 3), criterio de la rúbrica afectado, texto íntegro de la observación, decisión del equipo, y el o los commits en los que la resolución quedo aplicada, con el hash corto. Los mensajes de commit referencian el código (OBS-XX).
- El repositorio conserva las etiquetas `v0.7.0`, `v0.7.1` y `v0.9.0-rc` intactas como fotografía de cada entrega, y coloca la etiqueta `v1.0.0` sobre el commit final de la Entrega Final, con firma opcional GPG.
- El documento académico final incluye, en su Anexo A, la tabla completa de observaciones acumuladas con el porcentaje total resuelto por entrega y las razones de las no resueltas si las hubiera.

2.2. Bloque A — Cierre del producto software v1.0.0

El sistema debe alcanzar el estado estable declarado en el planteamiento original del PFC. Los requisitos se organizan en dos apartados: A.1 recoge las propiedades funcionales y de calidad heredadas de las Entregas 2 y 3, y A.2 consolida la estrategia de acceso a datos con la evidencia empírica cuantitativa que la Entrega Final exige.

A.1 — Propiedades funcionales y de calidad

- Todos los requisitos de las Entregas 2 y 3 operativos, sin regresiones.
- Cobertura JaCoCo (*lines* y *branches*) igual o superior al 70 % en los módulos de dominio, servicios y controladores de la API. El reporte HTML y XML se archiva en `docs/mediciones/jacoco/` con fecha ISO 8601 [40].
- Prueba de carga con k6 en configuración 50 VUs durante 30 s, con al menos cinco corridas independientes archivadas, contra el *endpoint* de listado protegido. Se cumple el umbral $p95 \leq 200\text{ms}$ con cache caliente y $p95 \leq 500\text{ms}$ con cache frio, sin errores HTTP ≥ 500 [17, 24].
- Auditoria Lighthouse con al menos tres corridas por perfil (*mobile*, *desktop*); umbrales cumplidos: *Performance* ≥ 80 , *Accessibility* ≥ 90 , *Best practices* ≥ 90 , *SEO* ≥ 90 .
- Los seis controles OWASP mínimos verificados con evidencia reproducible y, adicionalmente, un escaneo automático con una herramienta libre reconocida (por ejemplo *OWASP ZAP* en modo *baseline scan*), cuyo reporte HTML se archiva en `docs/mediciones/sec/zap/` [25].
- Etiqueta Git `v1.0.0` colocada sobre el commit de cierre, con firma opcional GPG. La imagen Docker publicada en *GitHub Container Registry* con *tag* `v1.0.0` y su *digest* `sha256` declarado en el README.
- El pipeline de CI (GitHub Actions) muestra al menos tres ejecuciones consecutivas exitosas con *badge passing* en el README.

A.2 — Consolidación de la estrategia híbrida de acceso a datos

La Entrega Final consolida y evidencia empíricamente la estrategia híbrida de acceso a datos introducida en la Tercera Entrega. La separación entre los CRUD elementales sobre entidades JPA y las operaciones adicionales encapsuladas en procedimientos almacenados o funciones SQL, definida en el apartado A.2 de la Guía de la Entrega 3, permanece íntegramente vigente y se refuerza con requisitos cuantitativos y de trazabilidad propios de la versión estable [9, 38, 12]. La legitimidad técnica de este enfoque híbrido está respaldada por la norma OWASP para prevención de inyección SQL, que reconoce ambos mecanismos (ORM parametrizado y procedimientos almacenados correctamente parametrizados) como defensas primarias equivalentes [26].

A.2.1 — Umbral cuantitativo mínimo de procedimientos almacenados

- El sistema v1.0.0 debe contener al menos **seis procedimientos almacenados o funciones SQL** versionados en `db/procs/`, uno por cada categoría funcional del apartado A.2.2 de la Guía de la Entrega 3 (consultas multi-tabla, cálculos agregados, reportes, actualizaciones masivas, validaciones cruzadas y generación de códigos secuenciales). Si el dominio del PFC no requiere alguna de las categorías, el equipo justifica la ausencia en el capítulo de Diseño del documento académico.
- Cada procedimiento y función se invoca desde código Java o del lenguaje elegido mediante los mecanismos formales de la especificación JPA 2.1 (`@Procedure` sobre método de repositorio Spring Data o `@NamedStoredProcedureQuery` sobre entidad); queda prohibido invocarlos mediante concatenación de cadenas en `createNativeQuery(...)` [9, 38].
- El catálogo `docs/basedatos/CATALOGO-SP.md` lista todos los procedimientos y funciones con nombre, categoría funcional, propósito, parámetros de entrada y salida con tipo y modo (IN, OUT, INOUT), cursores devueltos y tablas afectadas.

A.2.2 — ADR-006 y trazabilidad en la matriz

- El repositorio contiene un ADR-006 formal siguiendo la plantilla de Nygard con título *Estrategia de acceso a datos: separación CRUD/SP*, que documenta la decisión, sus alternativas descartadas (todo en ORM, todo en SP, patrón CQRS explícito) y sus consecuencias [23].
- La matriz de trazabilidad de `docs/trazabilidad/matriz.csv` incluye una columna adicional `tipo_acceso` con valores `CRUD-ORM` o `SP` para cada requisito funcional; cada fila trazada a un procedimiento almacenado referencia el archivo SQL correspondiente en `db/procs/`.

A.2.3 — Evidencia de auditoría automática sobre concatenación

- El pipeline de CI ejecuta una regla estática que rechaza cualquier concatenación sospechosa en el código fuente. Para pilas Java se acepta un *spotbugs* con *find-security* configurado con la regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE` y sus variantes, o un analizador equivalente en la pila elegida. El reporte de la última

ejecución se archiva en `docs/mediciones/sec/static-analysis/`.

- El pipeline ejecuta además una revisión automática de los archivos `db/procs/*.sql` que rechaza el uso de `EXECUTE IMMEDIATE`, `sp_executesql` o equivalentes con expresiones construidas por concatenación. La regla se implementa como *shell script* versionado bajo `scripts/audit-sql-dynamic.sh`.

A.3 — Culminación de la ingeniería de requisitos

La ingeniería de requisitos consolidada en la Entrega 3 se cierra en la Entrega Final con la validación final del SRS contra el sistema entregado. El proceso completo debe seguir las cuatro fases clásicas de la ingeniería de requisitos identificadas por Pohl (elicitación, negociación, documentación y validación) y por Wiegers y Beatty (desarrollo y gestión), y debe cumplir estrictamente con los procesos de ciclo de vida definidos en la norma *ISO/IEC/IEEE 29148:2018* [16, 30, 41].

A.3.1 — SRS final validado

- El SRS de la Entrega 3 se actualiza a `docs/requisitos/SRS-v1.0.0.pdf`, con la firma de aprobación (física o electrónica) del docente-director. La versión anterior queda archivada en `docs/requisitos/historico/`.
- Todos los requisitos con prioridad *Must* están en estado **verificado** en la matriz de trazabilidad. Los requisitos *Should* están en estado **implementado** o **verificado**. Los requisitos *Could* pueden permanecer en estado **pendiente**, siempre justificados en el capítulo de trabajo futuro del documento académico.
- Cada requisito individual satisface las nueve características de calidad para *requisitos individuales* de INCOSE v4 (C1–C9): *Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct* y *Conforming*. El conjunto completo satisface las seis características de calidad para *conjuntos de requisitos* (C10–C15): *Complete, Consistent, Feasible, Comprehensible, Able to be validated* y *Correct*. El equipo aporta como anexo del documento académico la lista de verificación INCOSE completa [15].
- El equipo declara métricas de calidad de requisitos: número total de requisitos, distribución por tipo, porcentaje verificado, número de cambios registrados en `docs/requisitos/CHANGELOG-REQ.md`, y tasa de estabilidad de requisitos calculada como uno menos la razón entre requisitos modificados y requisitos totales.

A.3.2 — Historias de usuario y casos de uso finales

- El conjunto final de historias de usuario, en formato Connextra con criterios INVEST y acceptance criteria en Gherkin, se archiva bajo `docs/requisitos/historias/`. Cada historia está trazada al menos a una prueba automatizada de aceptación [8].
- El conjunto final de casos de uso, en la plantilla de Cockburn niveles 1–4, se archiva bajo `docs/requisitos/casos-de-uso/`. Cada caso está trazado a un flujo del diagrama de secuencia y a al menos una prueba de integración [7].

A.4 — Puesta en producción del sistema

El sistema v1.0.0 debe estar desplegado en un ambiente publicamente accesible durante toda la semana de defensa, para que el tribunal pueda verificarlo en tiempo real durante la evaluación oral y para que un revisor externo pueda validar las propiedades reportadas.

A.4.1 — Ambiente de producción accesible

- El sistema esta desplegado en al menos uno de los siguientes tipos de ambiente: (i) un *Virtual Private Server* (VPS) con dominio DNS asignado; (ii) un proveedor de contenedores gratuito o académico (Fly.io, Railway, Render, Vercel para el frontend); (iii) un proveedor *cloud* con cuenta gratuita o de estudiante (Oracle Cloud Free Tier, AWS Free Tier, Google Cloud Free Tier); (iv) un servidor institucional facilitado por la UTEQ; o (v) una combinación equivalente. En cualquier caso, la URL publica esta declarada en el README y en la portada del PDF final.
- El sistema en producción sirve el frontend por HTTPS con certificado valido de una autoridad reconocida (por ejemplo Let's Encrypt), sin advertencias de seguridad del navegador. El endpoint de la API responde por HTTPS con el mismo certificado o con uno propio equivalente.
- El endpoint `/actuator/health` devuelve estado UP en todos los componentes registrados (base de datos, Redis, sistema de archivos si aplica).
- Existe un usuario *demo* preconfigurado, con credenciales publicadas en el README, para que el tribunal pueda iniciar sesion sin necesidad de registrar una cuenta.
- El acceso al sistema en producción se garantiza al menos desde el commit de tag v1.0.0 hasta 30 días después de la defensa.

A.4.2 — Documentación del despliegue

- El archivo `docs/despliegue/DEPLOYMENT.md` describe el proveedor, los recursos consumidos (CPU, RAM, disco), la topologia de red simplificada, las variables de entorno de producción (sin valores sensibles), y el procedimiento paso a paso para reproducir el despliegue.
- El archivo `docs/despliegue/RUNBOOK.md` contiene el manual de operación básica: procedimiento de arranque y apagado ordenado, procedimiento de rotacion de secretos (JWT, contraseñas de base de datos), procedimiento de rotacion de contenedores por actualizaciones de seguridad, y procedimiento de restauracion desde respaldo.
- El archivo `docs/despliegue/BACKUP.md` describe la estrategia de respaldo: frecuencia, destino, retencion, procedimiento de restauracion y prueba periodica de restauracion documentada. Como mínimo, se conserva un respaldo diario de la base de datos durante los treinta días posteriores a la defensa.
- El repositorio incluye un ADR-007 sobre la estrategia de despliegue elegida (aunque los otros seis ADR ya cubren la mayor parte de las decisiones, esta es lo suficientemente relevante para tener su propio decisión record).

2.3. Bloque B — Documento tecnico academico final del PFC

Este bloque es el que caracteriza a la Entrega Final y la distingue de las entregas previas. El equipo redacta un documento técnico académico completo (35–60 páginas de contenido, sin contar anexos), en formato PDF, siguiendo el patron IMRaD (*Introduction, Methods, Results and Discussion*) ampliado con las declaraciones obligatorias que hoy exigen las revistas científicas de alto impacto en ingeniería de software y disciplinas afines [31, 36, 27]. La estructura de secciones que se detalla a continuación es *obligatoria*. La ausencia de cualquier sección o su tratamiento superficial se penaliza directamente en la rúbrica.

B.1 — Portada institucional

Portada UTEQ con el título del PFC, nombres completos de los integrantes con su afiliación (UTEQ, FCI, Carrera de Software), identificador ORCID de cada integrante (obligatorio; obtener registro gratuito en orcid.org), nombre y correo del docente-director, semestre y periodo académico, fecha de cierre, etiqueta `v1.0.0` y *commit hash* corto del cierre, y DOI persistente del artefacto en Zenodo [13, 10, 37].

B.2 — Resumen estructurado en español e inglés

Dos resúmenes de 200 a 250 palabras cada uno, con estructura obligatoria en cinco elementos: *Contexto y problema*, *Objetivo*, *Metodos*, *Resultados principales*, *Conclusiones*. Un tercero debe poder decidir con solo este resumen si el trabajo le interesa. Sigue a cada resumen una lista de cinco a ocho *palabras clave* controladas: al menos tres deben coincidir con términos aceptados por el *ACM Computing Classification System 2012* o por la taxonomía *IEEE Thesaurus*. La versión en inglés no es una traducción literal: se redacta de forma nativa en inglés técnico correcto.

B.3 — Indices y lista de siglas

Índice general, índice de figuras, índice de tablas, índice de listados de código, y lista de siglas y acrónimos con expansión completa en su primera aparición.

B.4 — Capítulo 1: Introduccion

Contiene cinco subsecciones obligatorias.

- *Contexto y motivacion*: descripción del problema real que resuelve el sistema, con datos cuantitativos que dimensionen su magnitud (población afectada, tiempo o costo actual del problema, etc.). Se aportan al menos tres referencias que sustenten cuantitativamente el problema.
- *Preguntas de investigacion*: dos a cuatro preguntas de investigación (RQ1, RQ2, ...) formuladas en interrogativas cerradas y medibles. Estas preguntas guían el resto del documento.
- *Objetivo general y objetivos especificos*: un objetivo general y tres a cinco específicos formulados con verbo en infinitivo. Cada objetivo específico se traza contra al menos una

pregunta de investigación.

- *Contribuciones*: lista numerada de las contribuciones concretas del trabajo (por ejemplo: un sistema de ..., una evaluación empírica de ..., un conjunto de datos publico con DOI, un paquete reproducible). Cada contribución se traza a un componente concreto del repositorio [28].
- *Organizacion del documento*: parrafo único que anuncia lo que trata cada capítulo posterior.

B.5 — Capítulo 2: Marco teórico

Presentacion de los fundamentos conceptuales estrictamente necesarios para comprender el trabajo. No es un compendio general de teoría de aplicaciones web. Se limita a: fundamentos de ingeniería de requisitos según ISO/IEC/IEEE 29148:2018 y el marco de Pohl, con énfasis en el proceso de elicitation, análisis, especificación y validación [16, 30, 41]; características de calidad de un requisito bien formado según INCOSE v4 [15]; historias de usuario y casos de uso como técnicas complementarias de especificación [8, 7]; modelo arquitectónico C4 y su justificación [6]; características de calidad de ISO/IEC 25010 relevantes al proyecto [17]; principios REST [11]; esquema de autenticación elegido y su fundamento (JWT según RFC 7519, con las razones para colocarlo en cookie *HttpOnly*) [18]; controles OWASP Top 10 abordados [25, 24]; patrones de acceso a datos y modelos de consistencia relevantes [20, 4, 32]; y todas las nociones adicionales que el equipo requiera. Cada concepto invocado se cita a una fuente autoritativa.

B.6 — Capítulo 3: Trabajos relacionados

Busqueda estructurada de trabajos previos sobre problemas comparables al del PFC. La busqueda se documenta siguiendo los principios de una revisión sistemática adaptada a la escala del PFC. Se sigue la orientacion metodologica de Kitchenham y Charters, con la reduccion natural correspondiente a un PFC, y se reporta con la disciplina de PRISMA 2020 [19, 27]. En particular:

- Se declara la estrategia de busqueda: bases de datos consultadas (por ejemplo IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect, Springer Link), cadena de busqueda con operadores booleanos y sinonimos, ventana temporal, criterios de inclusion y exclusion.
- Se presenta un diagrama de flujo estilo PRISMA 2020 con el número de registros identificados, cribados, elegibles e incluidos, y las razones de exclusion [27].
- Se sintetizan los trabajos incluidos en una *tabla comparativa* con columnas: referencia, ano, dominio, pila tecnológica, patrones arquitectonicos, evaluación empírica reportada, limitaciones declaradas y diferencia frente a este trabajo. La tabla contiene un mínimo de ocho filas.
- Se cierra el capítulo con un parrafo de *brecha identificada* (*research gap*) donde se explícita en que aporta el trabajo del PFC respecto de lo publicado.

B.6bis — Capítulo 4: Ingeniería de requisitos

Este capítulo es propio y obligatorio del documento académico del PFC, y no debe fusionarse con el capítulo de diseño. Documenta el proceso completo de ingeniería de requisitos que llevo a la especificación vigente en la versión v1.0.0, con la disciplina y las plantillas de la norma *ISO/IEC/IEEE 29148:2018* y del marco de Pohl [16, 30]. El capítulo contiene:

- *Contexto y stakeholders*: identificación de todos los interesados (usuarios finales, patrocinador, mantenedor, regulador, competidor, etc.) siguiendo la técnica de *stakeholder onion* de Alexander o equivalente [33, 41].
- *Técnicas de elicitation aplicadas*: se declaran explícitamente las técnicas usadas (entrevistas semi-estructuradas, observación, prototipado, workshops, análisis de documentos, cuestionarios) con evidencia archivada bajo `docs/requisitos/elicitation/` (notas, guiones de entrevista, prototipos de baja fidelidad).
- *Requisitos funcionales y no funcionales* categorizados y priorizados MoSCoW, siguiendo el patron sintactico `[condition] [subject] shall [action] [object] [constraint]` de ISO/IEC/IEEE 29148:2018 y las 42 reglas de INCOSE v4 para redaccion de enunciados [16, 15].
- *Modelado de casos de uso y de historias de usuario*: diagrama de casos de uso UML de alto nivel, plantilla de Cockburn para los tres a cinco casos críticos y backlog de historias en formato Connextra con Gherkin acceptance criteria [7, 8].
- *Validacion de requisitos*: procedimiento aplicado (revisiones estructuradas, walkthroughs, prototipado evolutivo, criterios de aceptacion medidos contra pruebas automatizadas) y resultado de la validación final [30, 41].
- *Gestion del cambio de requisitos*: bitacora `docs/requisitos/CHANGELOG-REQ.md`, métricas de estabilidad (tasa de cambio, tasa de eliminación, tasa de agregacion) medidas entre Entrega 1A y Entrega Final.
- *Metricas de calidad de requisitos*: número total, distribucion por prioridad MoSCoW, distribucion por tipo (funcional, no funcional, restrictivo, de interfaz), porcentaje verificado, cobertura de trazabilidad hacia código y pruebas.

Este capítulo justifica que el PFC no es un ejercicio de desarrollo sin especificación previa, sino un artefacto disciplinado con corpus de requisitos verificable, valorable como evidencia primaria en revistas científicas dedicadas a la ingeniería de requisitos, como *Requirements Engineering* de Springer y como *Empirical Software Engineering* en la sección correspondiente [16, 31].

B.7 — Capítulo 5: Materiales y métodos

Se declara con precisión el enfoque metodologico del trabajo. Se adopta la metodología de *Design Science Research* de Peffers, articulada con los criterios de calidad de Hevner y con los estándares empíricos de la comunidad [28, 14, 31]. Según el tipo de evidencia empírica que el trabajo aporta

(estudio de caso único, evaluación empírica de rendimiento, evaluación de usabilidad), se acoge además la orientación metodológica correspondiente: Runeson y Höst para estudio de caso, Wohlin et al. para experimento controlado, Baltes y Ralph para el reporte del muestreo [34, 43, 2].

El capítulo describe:

- Enfoque metodológico global (DSR de Peffers en seis actividades) y su instanciación concreta.
- Instrumentos de medición: SUS de Brooke con los diez ítems originales sin modificación [5], k6 con *script* versionado, Lighthouse con configuración `lighthousec.js`, JaCoCo con umbrales, herramienta OWASP ZAP.
- Enfoque de métricas: se declara que las métricas del sistema están alineadas con un *Goal-Question-Metric* explícito [3]. Se presenta al menos un GQM completo (una meta, sus preguntas y sus métricas).
- Población, muestreo y participantes: se declara la población objetivo, la técnica de muestreo aplicada y sus limitaciones, siguiendo Baltes y Ralph [2]. Se justifica el tamaño de muestra elegido.
- Protocolo experimental replicable: cada bloque de evaluación (rendimiento, seguridad, usabilidad, cobertura, accesibilidad) se describe con precisión suficiente para que un tercero pueda reproducirlo.
- Análisis estadístico: se declaran de antemano los estadísticos que se calculan (media, mediana, desviación típica, percentiles p50, p90, p95, p99, intervalos de confianza al 95 %, tests inferenciales aplicables, tamaños de efecto).
- Determinismo y semillas: se declaran las semillas aleatorias usadas y las versiones exactas de las herramientas.

B.8 — Capítulo 6: Diseño del sistema y arquitectura

Diagramas C4 en niveles 1, 2 y 3, generados desde el código fuente Structurizr DSL versionado en el repositorio. Las seis ADRs se resumen en el capítulo (una por subsección) siguiendo la plantilla de Nygard, cubriendo los seis temas obligatorios del apartado D del Bloque D: elección de la pila principal, esquema de autenticación, gestor de base de datos, estrategia de cache, estrategia de despliegue y estrategia de acceso a datos (ADR-006, separación entre CRUD elementales sobre entidades ORM y operaciones adicionales encapsuladas en procedimientos almacenados o funciones SQL) [23, 6, 9]. Se incluye la tabla de atributos de calidad ISO/IEC 25010 con prioridad, escenario y estrategia; y la matriz de trazabilidad *requisito* → *módulo* → *endpoint* → *prueba* → *evidencia*, ahora con la columna adicional `tipo_acceso` que distingue accesos CRUD-ORM de accesos SP [17].

B.9 — Capítulo 7: Implementación

Se describe con precisión suficiente lo distintivo de la implementación: decisiones críticas de código (esquema de configuración, aislamiento de capas, manejo de errores globales, esquemas de *caching*, mecanismo de seguridad de la pila elegida, patrones concretos usados en el *frontend*). Se dedica además una subsección breve a la estrategia híbrida de acceso a datos, con al menos un listado de un procedimiento almacenado o función SQL representativo tomado de `db/procs/`, y el listado del método de repositorio que lo invoca mediante `@Procedure` o `@NamedStoredProcedureQuery`, siguiendo el contrato de la especificación JPA 2.1 y su implementación en Spring Data JPA [9, 38]. No se transcriben archivos completos; se remite al repositorio para el detalle. En total se ilustra con tres a seis listados de código pequeños y significativos, cada uno con su *caption* y su etiqueta, y todos referenciados desde el texto.

B.10 — Capítulo 8: Evaluación empírica y resultados

Presentación sistemática de los resultados obtenidos en cada bloque de evaluación, siempre con: (i) pregunta de investigación asociada; (ii) métrica reportada; (iii) datos crudos y su ubicación en el repositorio; (iv) estadística descriptiva con media, desviación típica e intervalo de confianza al 95 %; y (v) interpretación textual del resultado.

Requisitos mínimos en cada bloque:

- *Rendimiento (k6)*: cinco corridas independientes; tabla con `p50`, `p90`, `p95`, `p99`, `media`, `DT`, `IC 95 %`, `throughput`. Gráfico de barras con `IC 95 %` para `p95` en escenarios *cache-frio* y *cache-caliente*. Test inferencial (por ejemplo Wilcoxon de muestras pareadas) para la diferencia entre ambos escenarios, con tamaño de efecto (por ejemplo *Cliff delta*) [43].
- *Seguridad (OWASP + ZAP)*: tabla con los seis controles OWASP y su estado (aprobado / hallazgo menor / hallazgo mayor), y resumen del reporte ZAP con conteo por severidad [25].
- *Usabilidad (SUS)*: $N \geq 15$ participantes con perfil declarado (edad, sexo, experiencia web, dispositivo). Puntuación SUS media, DT e `IC 95 %`; interpretación según escala adjetival documentada en la literatura de SUS [5]. Diagrama de caja de las puntuaciones.
- *Cobertura (JaCoCo)*: tabla por módulo con cobertura de *lines* y *branches*; gráfico de barras.
- *Calidad web (Lighthouse)*: tabla con las cuatro categorías (Performance, Accessibility, Best practices, SEO) en perfiles *mobile* y *desktop*; media y DT sobre las tres corridas por perfil.

Todos los gráficos son vectoriales (SVG o PDF), generados desde los datos crudos por *scripts* versionados; su código fuente se referencia en el propio *caption*. Todas las paletas de color son accesibles a personas con daltonismo (por ejemplo *viridis* o *Okabe-Ito*).

B.11 — Capítulo 9: Discusión

Interpretación crítica de los resultados. Se responde una a una a cada pregunta de investigación planteada en el Capítulo 1, con evidencia concreta del Capítulo 7. Se compara el desempeño del

sistema frente a los trabajos relacionados del Capítulo 3 usando la misma tabla comparativa ampliada con los propios resultados. Se discuten hallazgos inesperados si los hubo. Se identifican implicaciones prácticas para el dominio del problema.

B.12 — Capítulo 10: Amenazas a la validez

Análisis explícito de amenazas a la validez según las cuatro categorías reconocidas en ingeniería de software empírica: validez de *constructo*, validez *interna*, validez *externa* y validez de *conclusion* [43, 34]. Para cada amenaza identificada se declara la mitigación aplicada. En particular, se aborda el sesgo de muestreo (siguiendo Baltes y Ralph) para la componente de usabilidad [2]. Este capítulo es imprescindible.

B.13 — Capítulo 11: Trabajo futuro

Tres a cinco líneas de trabajo futuro concretas, cada una en un párrafo dedicado, con justificación de por qué quedaron fuera del alcance actual y cómo se abordarían.

B.14 — Capítulo 12: Conclusiones

Síntesis final de una a dos páginas. Se responden brevemente las preguntas de investigación y se recapitulan las contribuciones enumeradas en la introducción, con las evidencias concretas del Capítulo 7.

B.15 — Declaraciones obligatorias

Al final del cuerpo del documento y antes de las referencias, con sección propia claramente etiquetada, se incluyen las declaraciones que hoy exigen las principales revistas y que consolidan la trazabilidad del trabajo [36, 21, 10].

- *Declaración de disponibilidad de código*: URL del repositorio público, etiqueta `v1.0.0`, DOI Zenodo del software.
- *Declaración de disponibilidad de datos*: DOI del *dataset* depositado en Zenodo (separado del software), licencia *Creative Commons Attribution 4.0* o equivalente, y ubicación de los datos crudos en el repositorio.
- *Declaración de disponibilidad de materiales*: colecciones Postman, *scripts* k6, `lighthouse.rc.js`, cuadernos de análisis, protocolo SUS, plantilla de consentimiento.
- *Declaración ética*: participantes en pruebas de usabilidad, consentimiento informado, tratamiento y anonimización de datos.
- *Contribuciones de autores según CRediT*: asignación explícita de los catorce roles de la taxonomía CRediT a cada integrante del equipo [21].
- *Declaración de conflictos de intereses*: los autores declaran ausencia de conflictos, o los declaran explícitamente.

- *Declaracion de financiamiento*: fuentes de financiamiento, incluida la ausencia de las mismas.
- *Declaracion de uso de asistentes de inteligencia artificial*: si el equipo utilizo asistentes generativos en cualquier fase del trabajo, se declara la herramienta, la versión, el propósito y la sección en que se uso.
- *Agradecimientos*: reconocimiento a personas o instituciones que colaboraron sin figurar como autores.

B.16 — Referencias y estilo

Bibliografía con un mínimo de **30 referencias** distintas, todas verificadas contra sus fuentes primarias. Al menos veinte de ellas son de alto impacto: articulos en revistas indexadas en *Scopus* o *Journal Citation Reports* (JCR) con cuartil Q1 o Q2, o articulos en actas de las principales conferencias del area (ICSE, FSE, ASE, MSR, EASE, ESEM). Toda referencia se cita en el cuerpo del documento; ninguna aparece solo en la lista final. El estilo de citas y referencias es APA septima edicion o IEEE, consistente en todo el documento. Cada cita respalda una afirmación no trivial concreta.

B.17 — Anexos

En anexos se incluyen: cadena de busqueda completa del Capítulo 3; protocolo detallado del ensayo SUS con las diez preguntas y la escala; capturas de la ejecución del pipeline CI con las tres corridas verdes; *checklist* completada del estándar empírico correspondiente de Ralph et al. 2021 según el tipo de estudio predominante [31]; y matriz de trazabilidad completa.

2.4. Bloque C — Evidencia empirica ampliada con analisis estadistico riguroso

Todo dato bruto que sustenta una figura o tabla del documento reposa en el repositorio bajo `docs/mediciones/` y se transforma en la figura o tabla mediante un *script* versionado bajo `scripts/`. La regla es exigente: no hay gráfico ni número en el documento que no sea regenerable con un solo comando desde los datos crudos, garantizando la reproducibilidad computacional en el sentido estricto de la comunidad científica [29, 35]. Esto habilita el nivel mas alto de reconocimiento en el modelo de badges de la ACM [1].

Requisitos estadísticos por bloque

- Todas las mediciones cuantitativas presentan media, desviación típica e intervalo de confianza al 95 %. Se prefieren intervalos de confianza sobre valores p como forma primaria de reporte.
- Cuando se comparan dos condiciones (por ejemplo, cache frio vs cache caliente), se reporta un test inferencial apropiado no parametrico por defecto (Wilcoxon, Mann-Whitney), acompañado siempre de tamaño de efecto (por ejemplo *Cliff delta*, r de rangos, *Cohen d*).
- Cuando se hacen comparaciones multiples, se declara y aplica la corrección correspondiente (por ejemplo Holm o Benjamini-Hochberg) y se documenta la decisión.

- Los gráficos usan paletas accesibles a personas con daltonismo. Cada gráfico tiene su script fuente referenciado en el *caption*.
- Las tablas usan líneas de `booktabs` (o equivalente en Word); no usan líneas verticales.

2.5. Bloque D — Reproducibilidad absoluta y publicabilidad del artefacto

La Entrega Final debe cumplir los criterios que la ACM identifica como *Artifacts Available + Artifacts Evaluated (Reusable) + Results Reproduced*, en su política vigente desde 2020 [1]. La lista siguiente concreta los requisitos que ese estado impone.

D.1 — Reproduccion end-to-end en un solo comando

- Objetivo `make all` que, desde una clonación limpia, ejecuta la secuencia completa: levantar contenedores, aplicar semillas, ejecutar pruebas, correr benchmarks, generar reportes, compilar el PDF del documento final. Si el comando termina con código cero, todo está reproducido.
- Imagen Docker publicada en *GitHub Container Registry* con *tag* `v1.0.0`; el *digest sha256* se declara en tres lugares (README, CITATION.cff, portada del PDF) y coincide entre los tres.
- Cuadernos ejecutables (*Jupyter* o *RMarkdown*) con salidas archivadas: al abrir un cuaderno se ve el último run que produjo los números del documento.

D.2 — Entorno declarado con precisión

- Version exacta de Docker, docker-compose, sistema operativo del contenedor, JDK, Node.js, Angular CLI y de cada herramienta usada, capturadas en `docs/entorno/versions.txt` generado por el pipeline y actualizado con cada release.
- Semillas aleatorias declaradas en cada script y en el README.

D.3 — Archivo permanente y metadatos completos

- Depósito en Zenodo del repositorio en el estado `v1.0.0`, obteniendo el DOI del software.
- Depósito en Zenodo del *dataset* de mediciones (`perf`, `sec`, `sus`, `lighthouse`, `jacoco`) en un registro separado con su propio DOI y su propia licencia (por ejemplo CC BY 4.0), siguiendo el principio de citación independiente de software y datos [37, 10].
- CITATION.cff `v1.2.0` con todos los campos: `cff-version`, `message`, `title`, `authors` (con ORCID de cada autor), `version`, `date-released`, `license`, `repository-code`, `doi`, `keywords`, `preferred-citation` [10, 13].

2.6. Bloque E — Cumplimiento de estándares de reporte

El equipo declara y evidencia el cumplimiento con los estándares de reporte pertinentes.

- *Estandar principal*: el estándar empírico ACM SIGSOFT correspondiente al tipo de estudio predominante del PFC (por ejemplo *Case Study*, *Controlled Experiment*, *Engineering Research*, *Data Science*). El *checklist* correspondiente se completa item por item y se archiva como anexo del documento y como `docs/checklists/ralph2021-<estandar>.md` en el repositorio [31].
- *Estandar complementario*: si el trabajo incluye una revisión de trabajos relacionados con procedimiento explícito, se completa el *checklist* PRISMA 2020 y se archiva como `docs/checklists/prisma2020.md` [27]. Si el trabajo es un estudio de caso, se documenta con los criterios de Runeson y Höst [34]. Si es un experimento controlado, con los de Wohlin et al. [43].
- *Directrices FAIR*: el paquete completo (software + datos + metadatos) cumple los cuatro principios F, A, I, R con checklist propio archivado en `docs/checklists/fair.md` [42].

2.7. Bloque F — Paquete de datos, materiales y metadatos

Se documenta y publica de manera separada el paquete de datos, respetando el principio de citación independiente de software y datos [37]. En el repositorio, la estructura `docs/mediciones/` contiene los datos crudos; la estructura `scripts/` contiene el código de análisis; los resultados agregados se generan al ejecutar el pipeline y no se comitean fijos, salvo para permitir la comparación con las corridas de referencia.

Se actualiza el `DATA-DICTIONARY.md` cubriendo el 100 % de las variables presentes en los archivos crudos, con nombre, tipo, unidad, rango y significado. Se añade un archivo `DATA-PROVENANCE.md` que describe, para cada tabla y figura del documento, exactamente que archivo crudo la origina, que script la produce y en que *commit hash* se generó por última vez.

2.8. Bloque G — Ética, consentimiento y disclosure

El bloque F de la Entrega 3 se conserva y se amplía. En particular:

- Los formularios de consentimiento firmados por los participantes SUS se archivan fuera del repositorio público, en un lugar declarado en `docs/etica/ETHICS.md` (por ejemplo *Google Drive* institucional del equipo, con acceso restringido al docente). Sus códigos anónimos P01, P02, ... aparecen en los datos crudos.
- Se declara en el documento final si algún asistente de IA generativa se utilizó, en qué fase, con qué propósito y con qué revisión posterior por parte del equipo.
- Se declara la ausencia de conflictos de interés y de financiamiento externo, o se declaran explícitamente en caso contrario.

3. Estructura obligatoria del repositorio para la Entrega Final

La estructura de la Entrega 3 se conserva y se amplía con los artefactos específicos de la Entrega Final. El árbol siguiente refleja el estado del repositorio en el tag v1.0.0.

Listing 1: Estructura mínima del repositorio para la Entrega Final

```
.
|-- README.md                # badges (CI, licencia, DOI software,
|                             # DOI dataset), arranque, credenciales
|-- LICENSE                  # OSI-approved (MIT o Apache-2.0)
|-- CITATION.cff             # 1.2.0 completo con ORCID y DOI
|-- CONTRIBUTORS.md          # roles CRediT explícitos por integrante
|-- CHANGELOG.md             # Keep-a-Changelog con v1.0.0
|-- Makefile                 # objetivos up down test bench audit
|                             # docs all clean
|-- docker-compose.yml       # digests sha256 pinados
|-- .env.example
|-- .gitignore
|-- backend/                 # Spring Boot 3.2.x (Java 21 LTS)
|-- frontend/                # Angular 17+
|-- db/
|   |-- schema.sql
|   |-- seed.sql
|   '-- procs/                # sp_*.sql y fn_*.sql versionados
|                             # (mínimo 6, uno por categoría)
|-- database/migrations/     # migraciones Flyway V1__, V2__, R__
|-- docs/
|   |-- informe-final.pdf    # documento académico completo
|   |-- informe-final.tex    # fuente LaTeX del documento
|   |-- refs.bib             # bibliografía >=30 refs verificadas
|   |-- requisitos/
|     |-- SRS-v1.0.0.pdf     # SRS final firmado ISO/IEC/IEEE 29148
|     |-- SRS-v1.0.0.tex    # fuente LaTeX del SRS
|     |-- historias/        # historias INVEST + Gherkin
|     |-- casos-de-uso/     # plantilla Cockburn niveles 1..4
|     |-- elicitation/      # evidencia de entrevistas y workshops
|     |-- CHANGELOG-REQ.md   # bitácora de cambios de requisitos
|     '-- historico/        # SRS-v0.3.0.pdf, SRS-v0.7.0.pdf, ...
|   |-- observaciones/
|     '-- OBSERVACIONES.md  # bitácora acumulada 1A + 1B + 3
|   |-- adr/                # 7 ADRs plantilla Nygard (incl.
|     |                     # ADR-006 acceso a datos CRUD/SP y
|     |                     # ADR-007 estrategia de despliegue)
|   |-- basedatos/
|     '-- CATALOGO-SP.md    # catálogo completo de SPs y funciones
|   |-- arquitectura/       # Structurizr DSL + PNG C4 L1..L3
|   |-- despliegue/
|     |-- DEPLOYMENT.md     # topología, proveedor, recursos
|     |-- RUNBOOK.md        # operación básica, rotación de secretos
|     '-- BACKUP.md         # respaldo y restauración
|   |-- mediciones/
|     |-- perf/             # k6 >=5 corridas independientes
```

```

| | |-- sec/
| | | |-- owasp/          # curl reproducible por control
| | | |-- zap/            # reporte HTML+XML ZAP baseline
| | | '-- static-analysis/ # reporte find-sec-bugs o equivalente
| | |-- sus/              # SUS raw csv N>=15
| | |-- lighthouse/      # JSON lhci por perfil mobile+desktop
| | |-- jacoco/           # report.xml + report.html
| | |-- DATA-DICTIONARY.md
| | '-- DATA-PROVENANCE.md
| |-- trazabilidad/
| | '-- matriz.csv        # req -> historia -> caso uso -> modulo
| |                        #   -> endpoint -> prueba ->
| |                        #   tipo_acceso -> evidencia -> estado
| |-- etica/
| | |-- ETHICS.md
| | |-- consentimientos/plantilla.md
| | '-- ai-disclosure.md   # uso de IA generativa (si aplica)
| |-- entorno/
| | '-- versions.txt       # docker, jdk, node, angular, k6, ...
| |-- checklists/
| | |-- ralph2021-<estandar>.md
| | |-- prisma2020.md      # si aplica
| | |-- runeson2009-case.md # si aplica
| | |-- wohlin2012-exp.md   # si aplica
| | |-- incose2023-req.md   # 15 características + 42 reglas
| | '-- fair.md
| |-- postman/coleccion.json
| '-- VERSIONING.md
|-- k6/                    # scripts + opts.js
|-- lighthouserc.js
|-- scripts/               # analisis con seed fija (Python o R)
| |-- perf-analysis.ipynb  # cuaderno con salidas ejecutadas
| |-- sus-analysis.ipynb
| |-- validate-traceability.sh # rechaza requisitos sin trazabilidad
| |-- audit-sql-dynamic.sh  # rechaza SQL dinamico en db/procs/*
| '-- gen-figuras.py
'-- .github/workflows/     # CI: test+bench+audit+ZAP+lhci+docs

```

4. Entregables consolidados de la Entrega Final

El equipo entrega, a través del repositorio Git publico y del formulario institucional del SGA:

1. Repositorio Git publico con *tag* exactamente igual a v1.0.0 y *commit hash* corto declarado en la portada del PDF final.
2. DOI persistente del software y DOI persistente del *dataset* (dos DOIs distintos, ambos en Zenodo).
3. PDF del documento académico final del PFC (35–60 páginas de contenido sin contar anexos) en docs/informe-final.pdf, con el código LaTeX fuente y su .bib en la misma

carpeta.

4. Colección Postman actualizada con al menos 25 peticiones cubriendo casos de éxito, validación, autorización y no encontrado.
5. Cuadernos de análisis Jupyter (o R Markdown) con salidas ejecutadas y visibles al abrir el archivo.
6. Video corto (5–7 minutos) demostrando `make all` desde una clonación limpia y mostrando los reportes generados. El video se aloja enlazado desde el README.
7. *Slides* para la defensa oral de la semana 17 (esta guía no cubre la defensa; ella se rige por la guía específica de la semana 17).

5. Rubrica de evaluacion de la Entrega Final

La rúbrica de la Entrega Final pondera diecisiete criterios agrupados en tres ejes: eje del *producto software* (criterios P0–P5, 35 %); eje del *documento tecnico academico* (criterios D0R y D1–D6, 40 %); y eje del *paquete de reproducibilidad, datos y publicabilidad* (criterios R1–R4, 25 %). Cada criterio se evalúa con cinco niveles: *Excelente* = 100 %, *Satisfactorio* = 75 %, *En desarrollo* = 50 %, *Insuficiente* = 25 %, *Ausente* = 0 %.

5.1. Formula de calificacion

$$\text{Nota}_{100} = \sum_{i=1}^{17} (\text{peso}_i \times \text{nivel}_i) \quad \text{Nota}_{10} = \frac{\text{Nota}_{100}}{10}$$

5.2. Rubrica detallada

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
EJE 1 — PRODUCTO SOFTWARE (35 %)					

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
P0. Aplicacion de observaciones acumuladas (Bloque 0)	5 %	docs/observaciones/observaciones.md completo con el 100 % de observaciones cerradas de las tres entregas anteriores (1A, 1B y 3); cada observación trazable a commit; tags v0.7.1 y v1.0.0 colocados correctamente; anexo A del documento académico con la tabla-resumen.	docs/observaciones/observaciones.md observaciones cerradas con trazabilidad correcta.	Entre 70 % y 85 % de observaciones cerradas o trazabilidad parcial.	Menos del 70 % cerradas o bitacora ausente.
P1. Cierre funcional, cobertura y estrategia de acceso a datos	7 %	Todos los requisitos operativos; JaCoCo lines+branches ≥ 70 % en dominio, servicios y controladores; CI con tres corridas verdes consecutivas; ≥ 6 procedimientos almacenados en db/procs/ invocados con @Procedure o @NamedStoredProc catalogados en docs/basedatos/CI y trazados en la matriz con columna tipo_acceso.	Cobertura ≥ 65 % en dos de tres capas; CI con al menos una corrida verde; ≥ 4 procedimientos almacenados catalogados.	Cobertura entre 50 y 65 %; CI con corridas rojas recientes; ≥ 2 procedimientos almacenados catalogados.	Cobertura < 50 %, CI sin corridas verdes, o menos de dos procedimientos almacenados pese a existir operaciones adicionales a los CRUD elementales.
P2. Rendimiento con evidencia estadística	6 %	≥ 5 corridas k6 con $p95 \leq 200$ ms caliente y ≤ 500 ms frio; IC 95 % reportados; test inferencial y tamaño de efecto.	≥ 3 corridas con umbrales cumplidos y estadística descriptiva completa.	≥ 3 corridas pero un umbral incumplido.	< 3 corridas o umbrales sin evidencia.

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
P3. Seguridad OWASP + ZAP + auditoria de acceso a datos	7 %	Seis controles OWASP evidenciados con curl reproducible; reporte ZAP baseline sin hallazgos altos; cabeceras HSTS, CSP, X-Frame-Options, XCTO presentes; análisis estático find-sec-bugs (o equivalente) sin hallazgos altos sobre concatenacion SQL; script <code>audit-sql-dynamic.sh</code> sin hallazgos en <code>db/procs/</code> .	Cinco controles OWASP y ZAP sin hallazgos altos; análisis estático presente con hallazgos menores documentados.	Cuatro controles OWASP evidenciados; análisis estático parcial.	Tres o menos controles evidenciados; sin análisis estático.
P4. Calidad web Lighthouse	5 %	Performance ≥ 80 , Accessibility ≥ 90 , Best Practices ≥ 90 , SEO ≥ 90 en perfiles mobile y desktop, con ≥ 3 corridas por perfil.	Tres de las cuatro categorías cumplen en ambos perfiles.	Dos categorías cumplen; corridas únicas.	Umbrales incumplidos en la mayoría.
P5. Puesta en produccion accesible (Bloque A.4)	5 %	Sistema desplegado en URL publica con HTTPS y certificado valido; <code>/actuator/health</code> en UP; usuario demo funcional; <code>DEPLOYMENT.md</code> , <code>RUNBOOK.md</code> y <code>BACKUP.md</code> completos; ADR-007 sobre estrategia de despliegue.	Sistema desplegado con HTTPS; documentación básica de despliegue presente; runbook o backup parcial.	Sistema desplegado sin HTTPS o con advertencias de certificado; documentación superficial.	No hay despliegue publicamente accesible o el sistema no responde.
EJE 2 — DOCUMENTO TECNICO ACADEMICO (40 %)					

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
D0R. Culminacion de la ingenieria de requisitos (Bloque A.3)	8 %	SRS final <code>docs/requisitos/SRS-final.docx</code> ; firmado por el docente-director; 100 % de <i>Must</i> verificados; historias y casos de uso finales trazados a pruebas; métricas de calidad de requisitos reportadas; cada requisito individual satisface las nueve características INCOSE v4 (C1–C9) y el conjunto satisface las seis características de conjunto (C10–C15); bitácora de cambios <code>CHANGELOG-REQ.md</code> continua.	≥ 90 % de <i>Must</i> verificados; métricas reportadas parcialmente; una a dos características INCOSE incumplidas de forma menor.	≥ 75 % de <i>Must</i> verificados; métricas reportadas de forma superficial; varias características INCOSE incumplidas.	SRS ausente, no firmado, o menos del 75 % de <i>Must</i> verificados.
D1. Estructura IMRaD, secciones obligatorias y capítulo de RE	6 %	Todas las 18 subsecciones obligatorias del Bloque B presentes y de calidad académica (incluye el capítulo B.6bis dedicado a la ingeniería de requisitos); resumen estructurado bilingüe con palabras clave controladas.	Una subsección faltante o tratada superficialmente.	Dos subsecciones faltante; resumen no estructurado.	Tres o mas subsecciones faltante.

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
D2. Trabajos relacionados y brecha	5 %	Estrategia de búsqueda declarada; diagrama PRISMA 2020; tabla comparativa con ≥ 8 trabajos; brecha explícita.	Estrategia y tabla presentes; diagrama parcial.	Trabajos citados pero sin estrategia estructurada.	Ausencia de estrategia y de brecha.
D3. Metodología y protocolo experimental	7 %	DSR Peffers instanciada; GQM presentado; muestreo justificado según Baltes y Ralph; estadística declarada a priori; semillas y versiones capturadas.	DSR instanciada; GQM parcial; muestreo declarado sin justificación completa.	Metodología mencionada pero sin instanciación clara.	Metodología ausente o genérica.
D4. Análisis estadístico riguroso	6 %	Media, DT, IC 95 % en todas las tablas; tests inferenciales con tamaño de efecto donde aplique; corrección múltiple documentada; SUS con $N \geq 15$.	IC 95 % en la mayoría de tablas; tests presentes en algunos casos; SUS con $N \geq 10$.	Solo estadística descriptiva básica; SUS con $N < 10$.	Sin estadística formal o SUS ausente.
D5. Amenazas a la validez y discusión crítica	4 %	Cuatro categorías (constructo, interna, externa, conclusión) tratadas con mitigación explícita; discusión responde una a una las RQs.	Tres categorías tratadas con mitigación; discusión responde la mayoría de RQs.	Dos categorías tratadas; discusión superficial.	Amenazas ausentes o superficiales.

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
D6. Bibliografía y calidad de citas	4 %	≥ 30 referencias distintas y verificadas; ≥ 20 de alto impacto (JCR Q1–Q2, ICSE, FSE, ASE, MSR, EASE, ESEM); todas citadas en el cuerpo; APA 7 o IEEE consistente.	≥ 25 referencias con ≥ 15 de alto impacto.	≥ 20 referencias con menos de 10 de alto impacto.	< 20 referencias o referencias fabricadas/no verificables.
EJE 3 — REPRODUCIBILIDAD, DATOS Y PUBLICABILIDAD (25 %)					
R1. Reproduccion end-to-end en un solo comando	8 %	make all regenera desde datos crudos hasta el PDF final sin intervencion; cuadernos con salidas ejecutadas; imagen Docker con digest declarado en 3 lugares.	make all regenera con una variable adicional documentada.	Regeneracion parcial requiriendo pasos manuales.	No hay regeneracion automática.
R2. Datos con DOI, diccionario y provenance	6 %	Dataset con DOI Zenodo separado del software y licencia CC BY 4.0; DATA-DICTIONARY.r 100 %; DATA-PROVENANCE.r traza cada tabla y figura al commit y script que la genera.	Dataset con DOI; diccionario ≥ 80 %; provenance parcial.	Dataset publicado sin DOI; diccionario parcial.	Sin DOI del dataset; diccionario ausente.
R3. Metadatos, CRediT y ORCID	6 %	CITATION.cff 1.2.0 completo con ORCID de todos los autores y DOI; CONTRIBUTORS.md con roles CRediT explicitos; declaraciones obligatorias completas.	CITATION.cff completo; roles CRediT parciales; falta una declaración.	CITATION.cff mínimo; ORCID solo de algunos autores.	Sin ORCID o sin CRediT.

Criterio	Peso	Excelente (100 %)	Satisfactorio (75 %)	En desarrollo (50 %)	Insuficiente (25 %)
R4. Cumplimiento de estándares de reporte	5 %	Checklist Ralph et al. 2021 del estándar principal completado item por item; checklist FAIR completo; checklist secundario según tipo de estudio.	Checklist principal completado; secundario parcial.	Un checklist parcial.	Sin checklists.
Total ponderado		100 % $\text{Nota}_{100} = \sum(\text{peso}_i \times \text{nivel}_i)$ $\text{Nota}_{10} = \text{Nota}_{100}/10$			

5.3. Reglas transversales de calificación

Reglas transversales que aplican a todos los criterios

1. **El repositorio Git es la fuente de verdad.** La evidencia verificada directamente en el repositorio se distingue siempre de la inferida del PDF o de capturas.
2. Los criterios del Eje 1 (P1–P4) y del Eje 3 (R1–R4) se califican *Ausente* (0 %) cuando no existe repositorio publico accesible o cuando el tag `v1.0.0` no existe o no coincide con el declarado en la portada del PDF.
3. Si `make all` falla desde una clonacion limpia siguiendo estrictamente el README, el criterio R1 se califica automáticamente *Insuficiente* (25 %) o menos, independientemente del resto.
4. Si el DOI del software declarado en el PDF no resuelve al tag `v1.0.0` en Zenodo, el criterio R3 no supera *En desarrollo* (50 %).
5. Si el documento académico no incluye la sección de *Amenazas a la validez* (D5), el criterio D5 se califica *Ausente* (0 %).
6. Cualquier referencia bibliografica fabricada, con datos inconsistentes o no verificable contra su fuente primaria descuenta un 25 % del criterio D6 por cada instancia detectada.
7. Cualquier evidencia de concatenacion de entrada de usuario en JPQL, HQL, SQL nativo o dentro de un procedimiento almacenado, y cualquier uso de SQL dinámico construido por concatenacion (`EXECUTE IMMEDIATE`, `sp_executesql` o equivalentes), califica automáticamente el criterio P1 como *Insuficiente* (25 %) y el criterio P3 como *Insuficiente* (25 %) o menos, independientemente del resto de la evidencia [26, 25].
8. Si la URL publica declarada en el README no responde en el momento de la evaluación, o el certificado HTTPS produce advertencias del navegador, el criterio P5 se califica automáticamente *Insuficiente* (25 %) o menos.
9. Si el SRS declarado en `docs/requisitos/SRS-v1.0.0.pdf` no contiene la aprobacion

del docente-director, o no cumple la plantilla de ISO/IEC/IEEE 29148:2018, el criterio D0R se limita a *En desarrollo* (50 %) como máximo [16].

10. Si la aplicación de observaciones acumuladas de las Entregas 1A, 1B y 3 no supera el 70 %, el criterio P0 se califica *Insuficiente* (25 %) y toda la Entrega Final se marca como *con deuda de calidad* en el reporte al SGA.
11. Si el pipeline CI muestra corridas rojas en los últimos siete días previos a la fecha de cierre declarada, el criterio P1 no supera *En desarrollo* (50 %).
12. Cualquier commit posterior al corte declarado en la portada del PDF se ignora para efectos de calificación, sin que ello penalice al equipo.
13. El documento académico debe estar redactado por los integrantes del equipo. Si se detecta reutilización no declarada de texto de otras fuentes por encima de umbrales académicos razonables, el criterio D6 se califica *Insuficiente* y se activa el protocolo institucional correspondiente.

6. Articulación con la defensa oral (semana 17)

La Entrega Final es la base sobre la cual el tribunal preparará sus preguntas durante la defensa oral de la semana 17. En consecuencia, la calidad del documento académico y del paquete de reproducibilidad determina en gran medida la fluidez del equipo durante la defensa. Se recomienda que cada integrante lea íntegramente el documento antes de la defensa, incluidas las secciones que no redactó directamente, y que el equipo prepare la demostración en vivo bajo la disciplina de **make all**: si esa demostración falla, el tribunal notará inmediatamente la fragilidad del artefacto. La preparación específica de la defensa (guion de presentación, distribución de tiempos, gestión de preguntas) se rige por la guía de la semana 17 y por el banco de preguntas frecuentes del tribunal, que se conserva del ciclo anterior de la asignatura.

Referencias

- [1] Association for Computing Machinery. Artifact review and badging — version 1.1, August 2020. Version 1.1 (August 24, 2020).
- [2] Sebastian Baltes and Paul Ralph. Sampling in software engineering research: a critical review and guidelines. *Empirical Software Engineering*, 27(4):94, 2022.
- [3] Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. The goal question metric approach. In John J. Marciniak, editor, *Encyclopedia of Software Engineering*, volume 1, pages 528–532. John Wiley & Sons, 1994.
- [4] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 4th edition, 2021.

- [5] John Brooke. SUS: A ‘quick and dirty’ usability scale. In Patrick W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor and Francis, London, 1996.
- [6] Simon Brown. *The C4 Model for Visualising Software Architecture*. Leanpub, 2023.
- [7] Alistair Cockburn. *Writing Effective Use Cases*. Agile Software Development Series. Addison-Wesley Professional, Boston, MA, 2001.
- [8] Mike Cohn. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional, Boston, MA, 2004.
- [9] Linda DeMichiel. JSR 338: Java(TM) persistence API, version 2.1 — final release. Technical Report JSR-338, Oracle America, Inc. — Java Community Process, May 2013. Final Release, 16 May 2013.
- [10] Stephan Druskat, Jurriaan H. Spaaks, Neil Chue Hong, Robert Haines, James Baker, Spencer Bliven, Egon Willighagen, David Pérez-Suárez, and Alexander Konovalov. Citation file format, 2021.
- [11] Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [12] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.
- [13] Laurel L. Haak, Martin Fenner, Laura Paglione, Ed Pentz, and Howard Ratner. ORCID: a system to uniquely identify researchers. *Learned Publishing*, 25(4):259–264, 2012.
- [14] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.
- [15] International Council on Systems Engineering. INCOSE guide to writing requirements. Technical Report INCOSE-TP-2010-006-04, International Council on Systems Engineering (INCOSE) — Requirements Working Group, July 2023.
- [16] International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 29148:2018, systems and software engineering — life cycle processes — requirements engineering. Technical Report ISO/IEC/IEEE 29148:2018, ISO/IEC/IEEE, November 2018.
- [17] ISO/IEC. ISO/IEC 25010:2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE). International Organization for Standardization, 2011.
- [18] Michael Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force (IETF), 2015.

- [19] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-001, Keele University and Durham University Joint Report, July 2007.
- [20] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [21] National Information Standards Organization (NISO). ANSI/NISO Z39.104-2022, CRediT, contributor roles taxonomy. Technical report, National Information Standards Organization, Baltimore, MD, February 2022.
- [22] Mark Nottingham and Erik Wilde. Problem details for HTTP APIs. RFC 7807, Internet Engineering Task Force (IETF), 2016.
- [23] Michael Nygard. Documenting architecture decisions. Cognitect Blog, November 2011.
- [24] Jose Manuel Ortega. *Mastering Python for Networking and Security: Leverage the scripts and libraries of Python version 3.7 and beyond to overcome networking and security issues*. Packt Publishing, 2nd edition, 2022.
- [25] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks, 2021.
- [26] OWASP Foundation. SQL injection prevention cheat sheet. OWASP Cheat Sheet Series, 2024. Documento vivo mantenido por la comunidad.
- [27] Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, Roger Chou, Julie Glanville, Jeremy M. Grimshaw, Asbjørn Hróbjartsson, Manoj M. Lalu, Tianjing Li, Elizabeth W. Loder, Evan Mayo-Wilson, Steve McDonald, Luke A. McGuinness, Lesley A. Stewart, James Thomas, Andrea C. Tricco, Vivian A. Welch, Penny Whiting, and David Moher. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, 372:n71, 2021.
- [28] Ken Peppers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77, 2007.
- [29] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011.
- [30] Klaus Pohl. *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, Heidelberg, 2010.
- [31] Paul Ralph, Nauman bin Ali, Sebastian Baltes, Domenico Bianculli, Jessica Diaz, Yvonne Dittrich, Neil Ernst, Michael Felderer, Robert Feldt, Antonio Filieri, Breno Bernard Nicolau de França, Carlo Alberto Furia, Gregory Gay, Nicolas Gold, Daniel Graziotin, Pinjia He, Rashina Hoda, Natalia Juristo, Barbara Kitchenham, Valentina Lenarduzzi, Jorge Martínez, Jorge Melegati, Daniel Mendez, Tim Menzies, Jefferson Molleri, Dietmar Pfahl, Romain

- Robbes, Daniel Russo, Nyyti Saarimäki, Federica Sarro, Davide Taibi, Janet Siegmund, Diomidis Spinellis, Mirosław Staron, Klaas Stol, Margaret-Anne Storey, Damian Tamburri, Marco Torchiano, Christoph Treude, Burak Turhan, and Sira Vegas. Empirical standards for software engineering research, 2021.
- [32] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020.
- [33] Suzanne Robertson and James Robertson. *Mastering the Requirements Process: Getting Requirements Right*. Addison-Wesley Professional, Boston, MA, 3 edition, 2012.
- [34] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009.
- [35] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013.
- [36] Iveta Simera, David Moher, John Hoey, Kenneth F. Schulz, and Douglas G. Altman. A catalogue of reporting guidelines for health research. *European Journal of Clinical Investigation*, 40(1):35–53, 2010.
- [37] Arfon M. Smith, Daniel S. Katz, Kyle E. Niemeyer, and FORCE11 Software Citation Working Group. Software citation principles. *PeerJ Computer Science*, 2:e86, 2016.
- [38] Spring Data Team. Spring data JPA reference documentation — stored procedures. VMware — Broadcom (Spring by VMware), 2025.
- [39] Craig Walls. *Spring in Action*. Manning Publications, 6th edition, 2022.
- [40] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.
- [41] Karl E. Wiegers and Joy Beatty. *Software Requirements*. Microsoft Press, Redmond, WA, 3 edition, 2013.
- [42] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016.
- [43] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2012.